

ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл холбооны технологийн сургууль



БИЕ ДААЛТЫН АЖЛЫН
ТАЙЛАН

Компьютерийн Хараа (F.CSM322)
2026 оны хичээлийн жилийн хавар

Шалгасан багш: Ч.Цэнд-Аюуш

Гүйцэтгэсэн: B221910002 Б.Насанжаргал
B221910005 Н.Цацрал

Улаанбаатар 2026

Хураангуй

Энэхүү тайланд webcam болон upload video дээр ажиллах real-time object detection, нүүрний landmark-д суурилсан AR filter, гарын landmark-д суурилсан interactive virtual drum mode, мөн study-space monitoring санааг нэгтгэсэн компьютерийн харааны веб системийн хөгжүүлэлтийг тайлбарлав. Төслийн гол зорилго нь зөвхөн загвар ажиллуулах бус, computer vision model-ийн гаралтыг хэрэглэгчид шууд мэдрэгдэхүйц тогтвортой, интерактив, ашиглахад ойлгомжтой систем болгон хувиргах явдал байв.

Систем нь backend талдаа FastAPI, OpenCV, Ultralytics YOLO болон ByteTrack tracking ашиглаж, frontend талдаа Next.js, React, TypeScript, Canvas 2D API, MediaPipe Face/Hand Landmarker болон Web Audio API ашигласан. Практик хэрэгжилтийн явцад detection jitter, latency, mirrored coordinate mapping, landmark alignment, event timing, audio scheduling зэрэг бодит real-time system-ийн асуудлуудыг шийдвэрлэсэн.

Туршилтын үр дүнгээр live camera болон video upload detection тогтвортой ажиллаж, temporal smoothing болон tracking хэрэглэсний дараа bounding box-ийн савлагаа багассан. Magic Mirror filter mode нь хоёр хүний нүүр дээр зэрэг filter байрлуулах боломжтой болсон бол Drum mode нь 6 төрлийн drum pad, beat challenge, score, combo, accuracy болон celebration effect бүхий интерактив demo болсон.

Түлхүүр үг

- Computer Vision
- Real-Time Object Detection
- YOLO
- ByteTrack
- MediaPipe
- Face Landmark Detection
- Hand Landmark Detection
- Augmented Reality
- Web Audio API
- Next.js
- FastAPI

Гарчиг

1. Оршил
2. Асуудлын тодорхойлолт ба төслийн зорилго
3. Системийн шаардлага
4. Ашигласан технологи
5. Системийн архитектур
6. Object detection ба tracking хэрэгжилт
7. Magic Mirror filter mode
8. Virtual drum mode ба rhythm challenge
9. Study space monitor
10. Гол алгоритмууд
11. Туршилт, үнэлгээ ба үр дүн
12. Хүндрэл, шийдэл ба эрсдэлийн үнэлгээ
13. Сурсан зүйл ба мэргэжлийн дүгнэлт
14. Цаашдын хөгжүүлэлт
15. Дүгнэлт
16. Ашигласан материал
17. Хавсралт

1. Оршил

Компьютерийн хараа нь зураг болон видео өгөгдлөөс объект, хөдөлгөөн, орон зайн байрлал, хүний нүүр болон гарын landmark зэрэг утга бүхий мэдээлэл гарган авч, түүнийг бодит хэрэглээний системд ашиглах боломж олгодог хиймэл оюун ухааны чухал салбар юм. Real-time computer vision системийн хувьд зөвхөн загварын ассурасу хангалттай биш; latency бага, frame хоорондын дүрслэл тогтвортой, хэрэглэгчийн интерфэйс ойлгомжтой байх шаардлагатай.

Энэхүү төсөл анх YOLO model ашиглан webcam болон video frame дээр объект илрүүлэх зорилготой эхэлсэн. Гэвч зөвхөн bounding box харуулах demo нь тайлангийн болон хамгаалалтын түвшинд хангалттай биш гэж үзсэн тул системийг илүү өргөн хүрээтэй интерактив платформ болгон өргөжүүлсэн. Үүний үр дүнд AI Magic Mirror хэсэг нэмэгдэж, хэрэглэгчийн нүүр болон гарын хөдөлгөөнийг ашигласан AR filter болон virtual drum тоглоомын горим хөгжүүлэгдсэн.

Төслийн онцлог нь backend дээр object detection/tracking, frontend дээр landmark detection, canvas rendering, audio feedback болон game logic зэрэг олон өөр чиглэлийн технологийг нэг системд уялдуулсанд оршино. Ийм бүтэц нь хичээлийн онолын мэдлэгийг бодит веб application болгон хэрэгжүүлэх сайн жишээ болсон.

2. Асуудлын тодорхойлолт ба төслийн зорилго

2.1 Асуудлын тодорхойлолт

Real-time object detection системийг шууд хэрэглэгчид үзүүлэх үед дараах асуудлууд нийтлэг гардаг: model-ийн inference хугацаа frame бүрийн rendering-тэй зөрөх, bounding box frame хооронд савлах, false positive түр хугацаанд гарч ирэх, selfie camera mirror хийсэн үед landmark coordinate буруу таарах, мөн интерактив үйлдэлд audio болон visual feedback хоцрох. Эдгээр асуудлыг шийдэхгүй бол model техникийн хувьд ажиллаж байгаа ч demo чанар муу, хэрэглэгчид найдваргүй мэт харагдана.

2.2 Ерөнхий зорилго

Энэхүү ажлын ерөнхий зорилго нь компьютерийн харааны хэд хэдэн аргыг нэгтгэсэн, real-time ажиллагаатай, хэрэглэгчтэй шууд харилцдаг веб application хөгжүүлэх явдал юм.

2.3 Тусгай зорилтууд

- Webcam болон upload video дээр real-time object detection хийх.
- YOLO detection-ийн гаралтыг tracking болон temporal smoothing ашиглан тогтвортой дүрслэх.
- MediaPipe Face Landmarker ашиглан нүүрний байрлал, хэмжээ, өнцөгт тохирсон AR filter ажиллуулах.
- MediaPipe Hand Landmarker ашиглан browser-side virtual drum mode хөгжүүлэх.
- Drum mode дээр beat challenge, timing judge, score, combo, accuracy болон celebration effect нэмж demo-г тоглоомын хэлбэрт оруулах.
- Frontend болон backend-ийн inference loop, rendering loop, API communication-ийг тогтвортой архитектураар зохион байгуулах.

3. Системийн шаардлага

3.1 Функциональ шаардлага

№	Шаардлага	Тайлбар
F1	Live camera detection	Webcam stream-ээс frame авч объект илрүүлэх.
F2	Video upload detection	Upload хийсэн video дээр detection overlay харуулах.
F3	Tracking	Frame хооронд объектын identity хадгалах.
F4	AR filter	Нүүрний landmark ашиглан filter зөв байрлуулах.
F5	Virtual drum	Гарын landmark ашиглан drum pad цохих үйлдэл илрүүлэх.
F6	Beat challenge	Дараалал, timing window, score, combo, accuracy тооцох.
F7	Study space analysis	Хүн болон эд зүйлсийн ойролцоо байдлыг хянах санааг хэрэгжүүлэх.

Хүснэгт 1. Системийн функциональ шаардлага

3.2 Функциональ бүс шаардлага

- Latency: хэрэглэгчийн camera болон audio interaction-д мэдэгдэхүйц хоцролт үүсгэхгүй байх.
- Stability: detection box frame бүр дээр хэт савлахгүй, false positive богино хугацаанд UI-г бохирдуулахгүй байх.
- Usability: demo хийх үед хэрэглэгч аль горимд юу хийх ёстойг шууд ойлгохуйц интерфэйстэй байх.
- Scalability: object detection, filter, drum, study-space зэрэг модулиудыг тусгаарлаж өргөтгөх боломжтой байх.
- Maintainability: backend endpoint, frontend loop, model logic, rendering logic-ийг тус тусад нь ойлгомжтой зохион байгуулах.

4. Ашигласан технологи

Давхарга	Технологи	Үүрэг
Backend	Python, FastAPI	API endpoint, frame processing, model inference orchestration
Computer Vision	OpenCV, NumPy	Image decode/encode, frame array representation, preprocessing
Object Detection	Ultralytics YOLO	Frame дээр объектын class, confidence, bounding box илрүүлэх
Tracking	ByteTrack	Объектын identity-г frame хооронд хадгалах
Frontend	Next.js 14, React, TypeScript	UI state, camera handling, page routing, type safety
Rendering	Canvas 2D API	Bounding box, filter overlay, drum pad, visual effect зурах
Landmark Detection	MediaPipe Tasks Vision	Face болон hand landmark detection-ийг browser талд ажиллуулах
Audio	Web Audio API	Drum sound synthesis, guide music,

Давхарга	Технологи	Үүрэг
		timing feedback
Styling	Tailwind CSS	Responsive, demo-friendly interface

Хүснэгт 2. Ашигласан технологийн үүрэг

5. Системийн архитектур

Системийг frontend болон backend гэсэн хоёр үндсэн хэсэгт хуваасан. Backend нь object detection болон tracking inference гүйцэтгэж, frontend нь camera/video stream авах, frame илгээх, response хүлээн авах, overlay зурах, landmark detection болон audio interaction-ийг удирдана. Энэ ялгалт нь server-side AI model болон browser-side interactive experience-ийг тусгаарлаж, хөгжүүлэлт болон туршилтыг илүү цэвэр болгосон.

[Зураг 1. Системийн ерөнхий архитектур]

User webcam/video -> Frontend capture loop -> FastAPI detect-frame endpoint -> YOLO/ByteTrack inference -> JSON metadata -> Canvas overlay rendering
MediaPipe face/hand landmark болон Web Audio API нь browser талд latency багатай ажиллана.

Зураг 1. Системийн ерөнхий архитектур



5.1 Backend flow

1. Frontend video frame-ийг canvas дээр зурж JPEG blob болгон хувиргана.
2. Frame-ийг /detect-frame endpoint рүү илгээнэ.
3. Backend OpenCV ашиглан image decode хийж NumPy array болгон боловсруулна.
4. YOLO model frame дээр object detection ажиллуулна.
5. Tracking идэвхтэй үед ByteTrack object ID үүсгэнэ.
6. Detection metadata-г JSON хэлбэрээр frontend рүү буцаана.
7. Frontend canvas дээр bounding box, label, confidence болон track ID-г дүрслэнэ.

5.2 Frontend dual-loop design

Frontend талд display loop болон detection loop-ийг салгаж зохион байгуулсан. Display loop нь requestAnimationFrame ашиглан video болон overlay-г тасралтгүй зурна. Detection loop нь тусдаа

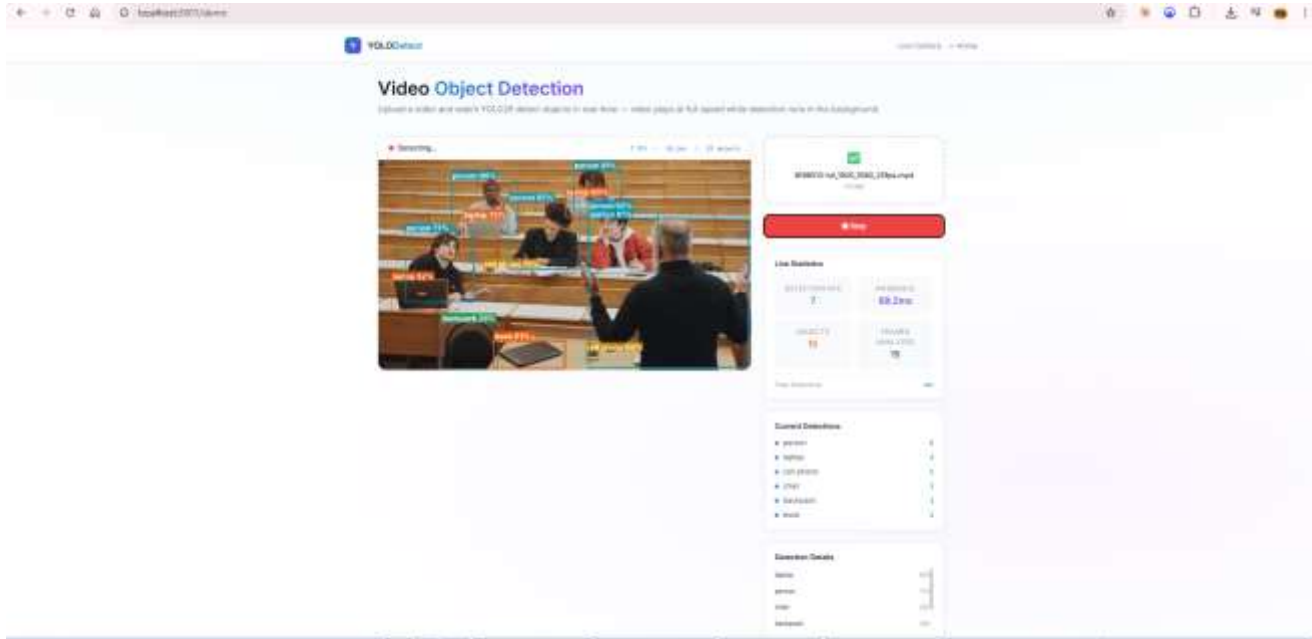
давтамжаар frame capture хийж backend рүү илгээнэ. Ингэснээр inference удаан болсон ч video playback зогсохгүй, хэрэглэгчийн нүдэнд UI илүү smooth харагдана.

[Зураг 2. Dual-loop frontend design]

Display loop: 60 FPS орчим canvas rendering.

Detection loop: backend inference response ирэх бүрт stabilized detection state шинэчлэгдэнэ.

Зураг 2. Dual-loop frontend design



6. Object detection ба tracking хэрэгжилт

6.1 YOLO detection

YOLO model нь нэг frame дээр олон объект илрүүлж, объект бүрийн class id, class name, confidence score болон bounding box координатыг буцаадаг. Энэхүү мэдээлэл нь UI дээр label болон rectangle хэлбэрээр харагдана. Системийн хувьд live camera болон upload video аль алинд нь ижил detection pipeline ашигласнаар code reuse болон consistency сайжирсан.

6.2 Temporal stabilizer

Raw detection box-уудыг шууд зурах үед frame бүр дээр координат бага зэрэг өөрчлөгдөж, хэрэглэгчийн нүдэнд box савлаж харагддаг. Тиймээс frontend талд temporal stabilizer нэмсэн. Stabilizer нь track_id байгаа үед тухайн ID-аар detection-ийг холбож, track_id байхгүй үед IoU matching ашиглан өмнөх frame-ийн box-той тааруулна. Дараа нь bounding box coordinate дээр EMA smoothing хэрэглэнэ.

Мөн нэг frame дээр түр гарсан сул detection-ийг шууд харуулахгүй байх, detection богино хугацаанд алга болсон ч box-ийг хэсэг hold хийх логик нэмж өгсөн. Үүний үр дүнд UI дээр илүү тогтвортой, demo хийхэд итгэлтэй харагдах visualization бий болсон.

6.3 Backend tracking

Backend дээр detect_and_track ашигласнаар ByteTrack object ID үүсгэж, frontend stabilizer-ийн matching илүү найдвартай болсон. Tracking нь нэг объект хөдөлж байсан ч frame хооронд нэг identity хадгалах тул box-ийн state management, smoothing, label consistency-д чухал үүрэгтэй.

7. Magic Mirror filter mode

7.1 Face landmark detection

Magic Mirror filter mode нь MediaPipe Face Landmarker ашиглан хэрэглэгчийн нүүрний landmark point-уудыг илрүүлнэ. Эдгээр point дээр үндэслэн нүдний шил, party hat, devil horns, сахал зэрэг AR filter-үүдийг нүүрний хэмжээ, байрлал, roll angle-тэй тааруулж canvas дээр зурна.

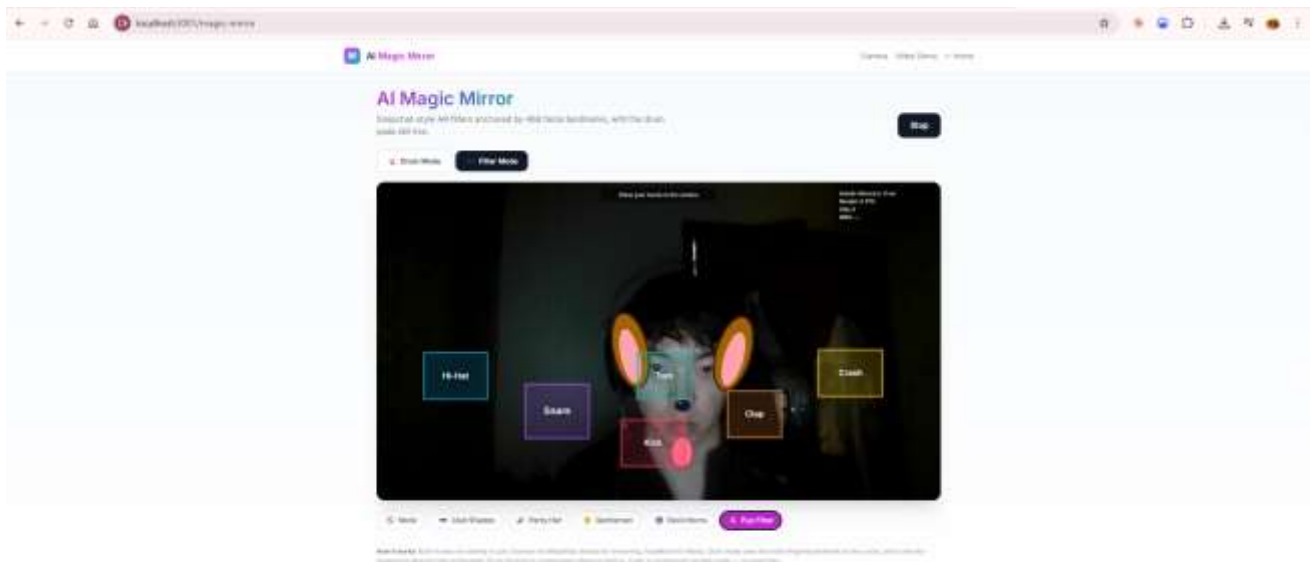
[Зураг 3. Face landmark ба filter alignment]

Нүдний шил: хоёр нүдний landmark-д тулгуурлана.

Сахал: хамар болон уруулын landmark ашиглана.

Party hat / horns: forehead landmark болон eye distance ашиглана.

Зураг 3. Face landmark ба filter alignment



7.2 Mirrored camera coordinate issue

Selfie camera view дээр video ихэвчлэн horizontal mirror хийгддэг. Энэ үед landmark coordinate болон screen coordinate хоорондын чиглэл зөрж, party hat болон horn filter буруу өнцгөөр эргэх асуудал гарсан. Үүнийг засахын тулд screen-left eye болон screen-right eye-ийг coordinate-аар тодорхойлж, roll angle-ийг үргэлж дэлгэцийн зүүнээс баруун тийш тооцдог болгосон. Ингэснээр filter-үүд нүүрний бодит байрлалтай зөв таарах болсон.

7.3 Олон хүний filter

Анхны хувилбар зөвхөн нэг нүүр илрүүлдэг байсан. numFaces тохиргоог нэмэгдүүлж, Face Landmarker-ээс ирсэн бүх face landmark array дээр loop хийснээр хоёр хүн camera дээр зэрэг орсон үед filter хоёр нүүр дээр зэрэг ажиллах боломжтой болсон. Энэ нь demo-ийн чанарыг мэдэгдэхүйц сайжруулсан.

8. Virtual drum mode ба rhythm challenge

8.1 Hand landmark detection

Drum mode нь MediaPipe Hand Landmarker ашиглан хэрэглэгчийн гарын landmark-ийг browser талд real-time илрүүлнэ. Index fingertip нь drum pad цохих cursor, wrist point нь гарын чиглэл болон visual guide line-д ашиглагдана. Browser-side detection ашигласнаар server round-trip latency багасч, audio feedback илүү natural мэдрэгддэг.

8.2 Predictive hit detection

Зөвхөн fingertip pad дотор орсон эсэхээр hit detection хийхэд false positive их гарах эрсдэлтэй. Тиймээс систем fingertip-ийн сүүлийн position-уудыг buffer-д хадгалж velocity тооцно. Хурдан доош хөдөлж байгаад огцом удааших буюу velocity reversal үүсэх үед drum hit гэж үзнэ. Хөдөлгөөн хэт хэвтээ чиглэлтэй бол hit гэж үзэхгүй. Мөн latency нөхөхийн тулд position-ийг бага зэрэг forward extrapolate хийдэг.

8.3 Web Audio API sound synthesis

Drum sound-уудыг audio file ашиглахгүйгээр Web Audio API-аар synthesize хийсэн. Kick sound нь sine oscillator болон pitch envelope, snare/hi-hat/crash/clap нь noise buffer болон filter, tom нь pitched oscillator envelope ашиглана. Энэ арга нь asset file багатай, browser дээр шууд ажиллах, timing нарийн хянах давуу талтай.

8.4 Beat challenge

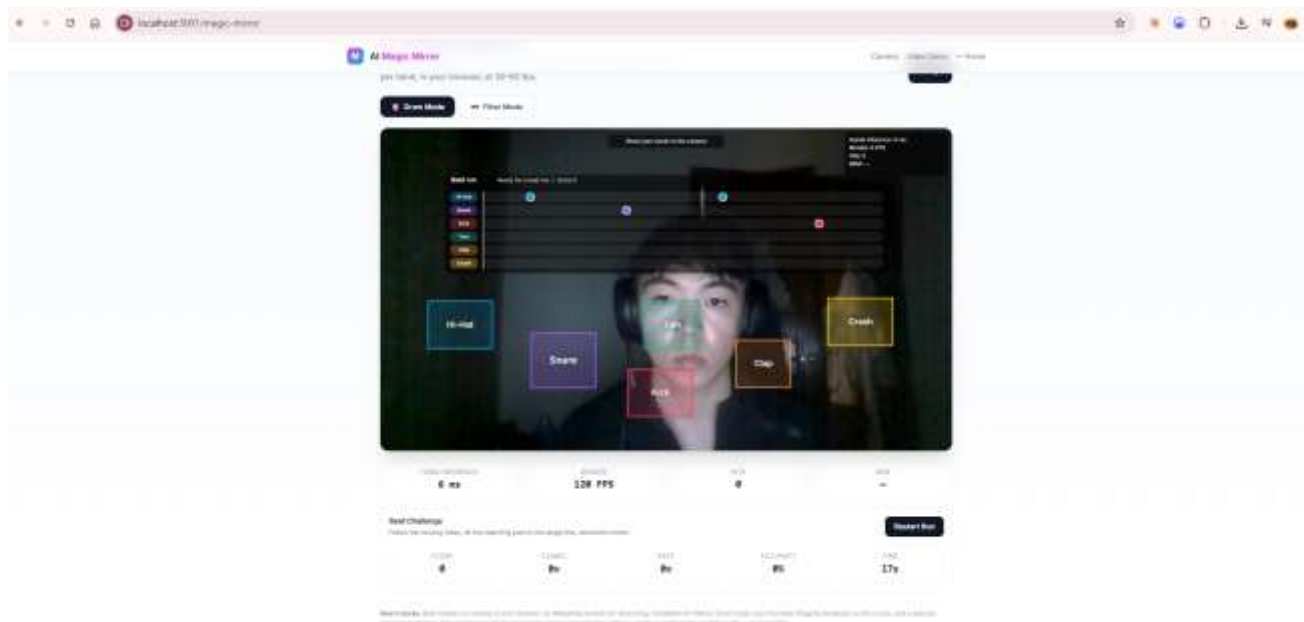
Демо-г илүү зорилготой болгохын тулд drum mode дээр rhythm-game маягийн beat challenge нэмсэн. Challenge нь хэрэглэгчид дараалал харуулж, зөв pad-ийг зөв timing window-д цохих даалгавар өгнө. Perfect, Great, Good, Miss гэсэн timing judge ашиглан score, combo, accuracy тооцно. Accuracy 50 хувиас дээш үед celebration fireworks effect гарч, demo дууссан мэдрэмжийг нэмэгдүүлдэг.

[Зураг 4. Drum mode UI ба beat challenge]

6 drum pad: Hi-Hat, Snare, Kick, Tom, Clap, Crash.

Timing judge: Perfect 75 ms, Great 140 ms, Good 220 ms, Miss > 220 ms.

Зураг 4. Drum mode UI ба beat challenge



9. Study space monitor

Study space monitor хэсэг нь номын сан эсвэл сургалтын орчинд хүн болон эд зүйлсийн ойролцоо байдлыг хянах санааг туршсан модуль юм. Энэ хэсэгт person, laptop, cell phone, book, backpack, bottle, cup зэрэг class-уудыг илрүүлж, эд зүйл тухайн хүнтэй ойр байгаа эсэхийг tracking болон state machine ашиглан дүгнэх боломжтой.

Одоогийн байдлаар энэ модуль нь prototype түвшинд боловч application-ийн бодит хэрэглээний чиглэлийг харуулж байна. Цаашид dashboard, alert logic, historical log, privacy-aware processing зэрэг хэсгийг нэмснээр сургалтын орчны monitoring систем болгон хөгжүүлэх боломжтой.

10. Гол алгоритмууд

10.1 Bounding box smoothing

Bounding box smoothing нь өмнөх frame-ийн box болон шинэ detection box-ийн хооронд exponential moving average ашиглан шилжилтийг зөөлрүүлдэг. Томъёогоор: $\text{smoothed_bbox} = \text{old_bbox} * (1 - \alpha) + \text{new_bbox} * \alpha$. Alpha хэт бага бол box хоцорч харагдана, хэт их бол jitter багасахгүй. Тиймээс real-time UI-д тохирох утгыг туршилтаар сонгосон.

10.2 IoU matching

Track ID байхгүй үед өмнөх frame-ийн box болон одоогийн frame-ийн box хооронд Intersection over Union буюу IoU тооцож нэг объект эсэхийг шийдсэн. $\text{IoU} = \frac{\text{intersection_area}}{\text{union_area}}$. IoU тодорхой босгоос их байвал өмнөх object state-тэй холбож smoothing үргэлжлүүлнэ.

10.3 Velocity reversal based hit detection

Drum hit detection нь fingertip position-ийн өөрчлөлтөөс velocity тооцож, хөдөлгөөн хурдан доош чиглэлтэй байгаад огцом удааших үед цохилт болсон гэж үзнэ. Энэ нь rad-д зүгээр орж гарах хөдөлгөөнөөс үүсэх false positive-г багасгаж, хэрэглэгчийн цохилтын мэдрэмжийг илүү бодит болгодог.

11. Туршилт, үнэлгээ ба үр дүн

Төслийг demo-driven байдлаар туршсан. Гол үнэлгээ нь live interaction-ийн тогтвортой байдал, detection overlay-ийн чанар, landmark alignment, drum hit detection-ийн мэдрэмж, rhythm challenge-ийн timing feedback дээр төвлөрсөн.

Туршилтын хэсэг	Шалгах зүйл	Ажиглагдсан үр дүн	Үнэлгээ
Live camera detection	Object detection, box rendering	Detection ажилласан; stabilizer нэмсний дараа box илүү тогтвортой болсон.	Амжилттай
Video upload detection	Playback ба detection loop	Video smooth тоглож, detection background-д шинэчлэгдсэн.	Амжилттай
Filter mode	Face landmark alignment	Sunglasses, party hat, devil horns зөв байрласан.	Амжилттай
Multi-face filter	2 хүний нүүр дээр filter	Face landmark array дээр loop хийснээр хоёр нүүр дээр зэрэг filter ажилласан.	Амжилттай
Drum mode	Hand tracking, hit detection, sound	6 drum sound ажиллаж, pad layout илүү зохион байгуулалттай болсон.	Амжилттай
Beat challenge	Score, combo, accuracy	Timing judge, guide music, celebration effect ажилласан.	Амжилттай
Study space	Object class filtering, ownership idea	Prototype түвшний detection болон state idea хэрэгжсэн.	Сайжруулах боломжтой

Хүснэгт 3. Туршилтын хураангуй

11.1 Үр дүнгийн үнэлгээ

Одоогийн төслийн хамгийн хүчтэй тал нь олон computer vision functionality-г нэг demo application-д нэгтгэсэн явдал юм. Харин судалгааны тайлангийн талаас харвал тоон хэмжилт буюу FPS, latency, average inference time, detection confidence distribution, hit detection accuracy зэрэг metric-үүдийг хэмжиж хариулах шаардлагатай.

12. Хүндрэл, шийдэл ба эрсдэлийн үнэлгээ

Хүндрэл	Шалтгаан	Хэрэгжүүлсэн шийдэл	Үлдэх эрсдэл
Real-time latency	Camera capture, model inference, rendering, audio playback зэрэг олон pipeline зэрэг ажиллах	Display loop болон detection loop салгасан; drum хэсэгт browser-side MediaPipe ашигласан	Сул төхөөрөмж дээр FPS буурах боломжтой
Detection jitter	YOLO frame бүр дээр бага зэрэг өөр box гаргадаг	Temporal smoothing, hold logic, ByteTrack tracking нэмсэн	Хэт хурдан хөдөлгөөн дээр box хоцрох боломжтой
Mirrored coordinate	Selfie camera болон landmark coordinate чиглэл зөрөх	Screen-left/right eye ашиглан angle тооцсон	Өөр camera orientation дээр дахин calibration хэрэгтэй
False drum hit	Pad дотор орох хөдөлгөөн	Velocity reversal, direction	Зарим гарын хөдөлгөөнд

Хүндрэл	Шалтгаан	Хэрэгжүүлсэн шийдэл	Үлдэх эрсдэл
	бүр hit болох магадлалтай	filtering, prediction ашигласан	false trigger үлдэх боломжтой
Audio scheduling	Guide music болон user hit зэрэгцэн ажиллах	Web Audio API oscillator, noise, envelope schedule ашигласан	Browser audio policy болон device latency нөлөөлнө

Хүснэгт 4. Хүндрэл, шийдэл, үлдэх эрсдэлийн үнэлгээ

13. Сурсан зүйл ба мэргэжлийн дүгнэлт

Энэ ажлаас computer vision model-ийг application-д оруулахад inference ажиллуулах нь зөвхөн эхний алхам юм. Хэрэглэгчид итгэл төрүүлэх demo хийхийн тулд model-ийн гаралтыг stabilization, tracking, UI state, latency control, interaction design зэрэг software engineering давхаргатай холбох шаардлагатай.

Мөн browser дээр MediaPipe ашиглах нь landmark detection-д server latency-г багасгаж, real-time interaction-д илүү тохиромжтой болохыг харуулсан. Canvas rendering нь object detection overlay, AR filter, game UI, visual effect зэрэг олон төрлийн дүрслэлийг нэг орчинд хэрэгжүүлэх боломж олгосон. Web Audio API нь audio asset ашиглахгүйгээр realtime feedback үүсгэхэд үр дүнтэй байсан.

14. Цаашдын хөгжүүлэлт

- FPS, inference latency, end-to-end latency, hit detection accuracy зэрэг quantitative metric хэмжих dashboard нэмэх.
- Mobile responsive layout болон touch/camera orientation calibration сайжруулах.
- Drum mode дээр difficulty сонголт, custom beat chart import, user performance history нэмэх.
- AR filter-ийн asset management, олон filter зэрэг ашиглах, face occlusion handling сайжруулах.
- Study space monitor дээр alert dashboard, privacy-aware mode, event log, ownership confidence score нэмэх.
- Detection stabilizer-ийн alpha, hold time, IoU threshold зэрэг параметрийг UI дээрээс тохируулах.
- Төслийн demo video, screenshot, GitHub repository structure, API documentation-ийг тайланд хавсаргах.

15. Дүгнэлт

Энэхүү бие даалтын ажлаар YOLO object detection, ByteTrack tracking, MediaPipe face/hand landmark, Canvas overlay, Web Audio API зэрэг олон технологийг нэгтгэсэн real-time computer vision web application-ийг бид хөгжүүлэв. Систем нь live camera detection, video upload detection, Magic Mirror filter mode, virtual drum mode, beat challenge болон study-space monitoring prototype гэсэн олон хэсэгтэй.

Төслийн гол үнэ цэнэ нь computer vision-ийн онолын мэдлэгийг интерактив хэрэглээний системд буулгасан явдал юм. Bounding box stabilization, multi-face filter, mirrored coordinate correction, predictive drum hit detection, guide music, score/accuracy logic зэрэг нэмэлтүүд нь төслийг энгийн demo-оос илүү мэргэжлийн түвшний системийн прототип болгон сайжруулсан. Цаашид тоон хэмжилт, хэрэглэгчийн туршилт, mobile support болон documentation-ийг нэмснээр илүү бүрэн хэмжээний судалгааны болон инженерийн тайлан болох боломжтой.

16. Ашигласан материал

- Ultralytics. YOLO documentation.
- Google MediaPipe. MediaPipe Tasks Vision documentation.
- FastAPI documentation.

11. OpenCV documentation.
12. MDN Web Docs. Canvas 2D API.
13. MDN Web Docs. Web Audio API.
14. React documentation.
15. Next.js documentation.
16. TypeScript documentation.

17. Хавсралт

17.1 Төслийг ажиллуулах үндсэн заавар

Backend ажиллуулах:

```
cd Biydaalt
python cli.py serve --port 8000
```

Frontend ажиллуулах:

```
cd Biydaalt/web
npm run dev
```

Browser дээр шалгах гол хаягууд:

```
http://localhost:3000
http://localhost:3000/camera
http://localhost:3000/demo
http://localhost:3000/magic-mirror
http://localhost:3000/study-space
```